

Airbus A350-900 | Polymorphie

Kategorie	Beschreibung
Definition von Polymorphie	Polymorphie ist das OOP-Prinzip, bei dem Objekte unterschiedlicher Klassen durch eine einheitliche Schnittstelle angesprochen werden können, wobei jede Klasse ihre eigene spezifische Implementierung der Methoden bereitstellt. Es ermöglicht eine flexible und erweiterbare Struktur, da die gleiche Methode auf Objekte unterschiedlicher Typen angewendet werden kann.
Anwendungsfall (Airbus A350-900)	Im Airbus A350-900 könnte Polymorphie verwendet werden, um verschiedene Systeme wie Engine , FlightControlSystem und CabinSystem durch eine einheitliche Schnittstelle anzusprechen, obwohl diese Systeme unterschiedliche Implementierungen der gleichen Methoden bereitstellen.
Ziel der Polymorphie	Flexibilität: Eine Methode kann für Objekte unterschiedlicher Typen aufgerufen werden, was den Code flexibler und erweiterbarer macht. Wiederverwendbarkeit: Durch die Verwendung von polymorphen Methoden können dieselben Methoden für verschiedene Objekttypen genutzt werden, ohne den Code zu duplizieren. Erweiterbarkeit: Neue Objekttypen oder Komponenten können durch die Implementierung von vorhandenen Schnittstellen hinzugefügt werden, ohne den bestehenden Code zu ändern.
Beispiel: Polymorphie bei Triebwerksystemen	Im Fall des Airbus A350-900 könnte es verschiedene Triebwerkstypen wie RollsRoyceTrentXWB und GE90 geben, die beide die Methode <code>start()</code> überschreiben, um den spezifischen Startvorgang zu implementieren. Ein FlightControlSystem könnte jedoch eine Liste von Engine -Objekten haben, und die Methode <code>start()</code> würde unabhängig vom Triebwerkstyp korrekt aufgerufen.
Vererbung und Polymorphie	Polymorphie wird oft in Verbindung mit Vererbung verwendet, da eine Methode in einer Superklasse definiert werden kann und von den Unterklassen auf spezifische Weise überschrieben wird. Zum Beispiel könnte eine Methode <code>adjustSpeed()</code> in der Superklasse AircraftSystem definiert werden, die dann von spezifischen Systemen wie FlyByWireSystem oder ManualControlSystem jeweils anders implementiert wird.
Beispiel: Polymorphie im Flugsteuerungssystem	Ein FlightControlSystem könnte ein polymorphes Verhalten aufweisen, bei dem die Methode <code>adjustAltitude()</code> je nach Systemtyp entweder durch manuelle Steuerung oder durch das FlyByWireSystem angepasst wird. Beide Systeme implementieren <code>adjustAltitude()</code> , aber auf unterschiedliche Weise.

Kategorie	Beschreibung
Dynamische Bindung	Polymorphie ermöglicht dynamische Bindung , was bedeutet, dass zur Laufzeit entschieden wird, welche Methode aufgerufen wird. Wenn ein AircraftSystem -Objekt zur Laufzeit ein spezifisches System wie Engine oder FlightControlSystem referenziert, wird zur Laufzeit entschieden, welche Methode ausgeführt wird, basierend auf dem tatsächlichen Objekttyp.
Beispiel: Polymorphie im Kabinensystem	Ein CabinSystem könnte polymorph sein, um mit verschiedenen Kabinenklassen (z.B. BusinessClassCabin , EconomyClassCabin) zu arbeiten. Die Methode <code>adjustTemperature()</code> könnte je nach Kabinenklasse unterschiedlich implementiert werden, aber das CabinSystem kann dennoch eine einheitliche Schnittstelle bieten, um die Temperatur anzupassen.
Methodenüberschreibung und Polymorphie	Polymorphie basiert häufig auf der Methodenüberschreibung (Method Overriding). Eine Methode, die in einer Superklasse definiert ist, wird in einer Unterklasse überschrieben, um das Verhalten spezifisch zu gestalten. Zum Beispiel könnte die Methode <code>calculateFuelEfficiency()</code> in der Superklasse Engine definiert werden und in einer Unterklasse RollsRoyceTrentXWB spezifisch für den Rolls-Royce-Triebwerkstyp überschrieben werden.
Vorteile der Polymorphie	Erhöhte Flexibilität: Durch die Möglichkeit, mit unterschiedlichen Objekttypen auf eine einheitliche Weise zu arbeiten, kann der Code flexibler gestaltet werden. Reduzierte Code-Duplikation: Die gleiche Methode kann für verschiedene Typen verwendet werden, was den Code übersichtlicher und wartungsfreundlicher macht. Erweiterbarkeit: Neue Typen und Klassen können einfach hinzugefügt werden, ohne dass der bestehende Code modifiziert werden muss.
Zusammenhang mit anderen OOP-Prinzipien	Abstraktion: Polymorphie und Abstraktion arbeiten zusammen, da die Verwendung von abstrakten Klassen und Schnittstellen es ermöglicht, polymorphe Methoden anzuwenden, ohne sich um die Details der Implementierung kümmern zu müssen. Vererbung: Polymorphie ist eng mit Vererbung verbunden, da es die Möglichkeit bietet, geerbte Methoden in Unterklassen zu überschreiben und spezifische Implementierungen bereitzustellen. Kapselung: Polymorphie und Kapselung arbeiten zusammen, um das System zu modularisieren. Objekte können über eine gemeinsame Schnittstelle angesprochen werden, ohne dass die Implementierungsdetails für den Benutzer zugänglich sind.